

Computer Networks Project

Private Network Service Platform

Phase 1 – Architecture Document

Group Name: dotenv

Team Members: Khushi 2401010225

Satvik Prasad 2401020055

1 Project Overview

Our Phase 1 implementation uses two Mac computers connected to the same private Wi-Fi/LAN network. The two physical machines combine multiple logical roles.

Mac 1 provides:

- Primary DNS using `dnsmasq`
- Backend A
- Client functionality

Mac 2 provides:

- nginx reverse proxy and load balancer
- TLS termination
- Backend B
- Client functionality

The Phase 1 request path is:

Client → **DNS** → **TCP** → **TLS** → **nginx** → **Backend A** / **Backend B** → **nginx** → **Client**

2 Network Topology Diagram

The Phase 1 system is implemented using two physical Mac computers connected to the same private Wi-Fi/LAN network. Multiple logical roles are combined on these two machines.

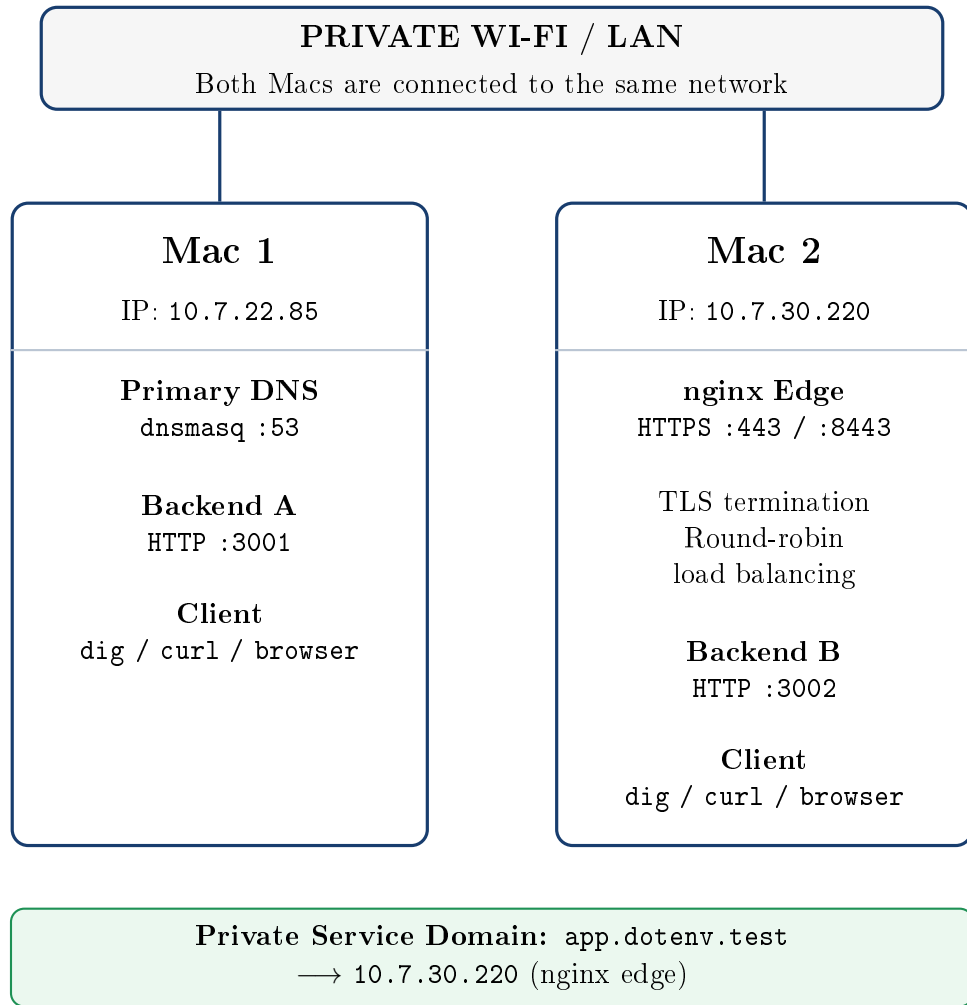
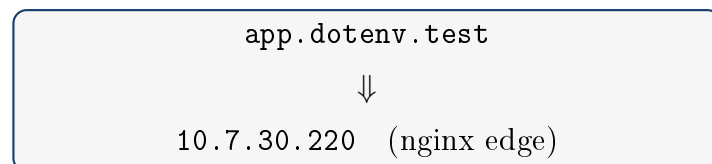


Figure 1: Phase 1 Two-Mac Private Network Topology

2.1 Service Domain Resolution

The private service domain is resolved by the internal DNS server and points to the nginx edge machine.



The DNS relationship is therefore:

`app.dotenv.test → 10.7.30.220`

This ensures that clients connect to nginx rather than directly to either backend.

2.2 Machine Roles and IP/Service Table

Machine	Role	IP Address	Services / Ports
Mac 1	Primary DNS + Backend A + Client	10.7.22.85	dnsmasq : 53 Backend A : 3001
Mac 2	nginx Edge + Backend B + Client	10.7.30.220	nginx HTTPS : 443 / 8443 Backend B : 3002

Table 1: Machine Roles, IP Addresses, and Services

3 Request Flow Diagram

The following diagram shows the complete path of a client request, including DNS resolution, TCP connection establishment, TLS negotiation, nginx reverse proxying, load balancing, and the backend response.

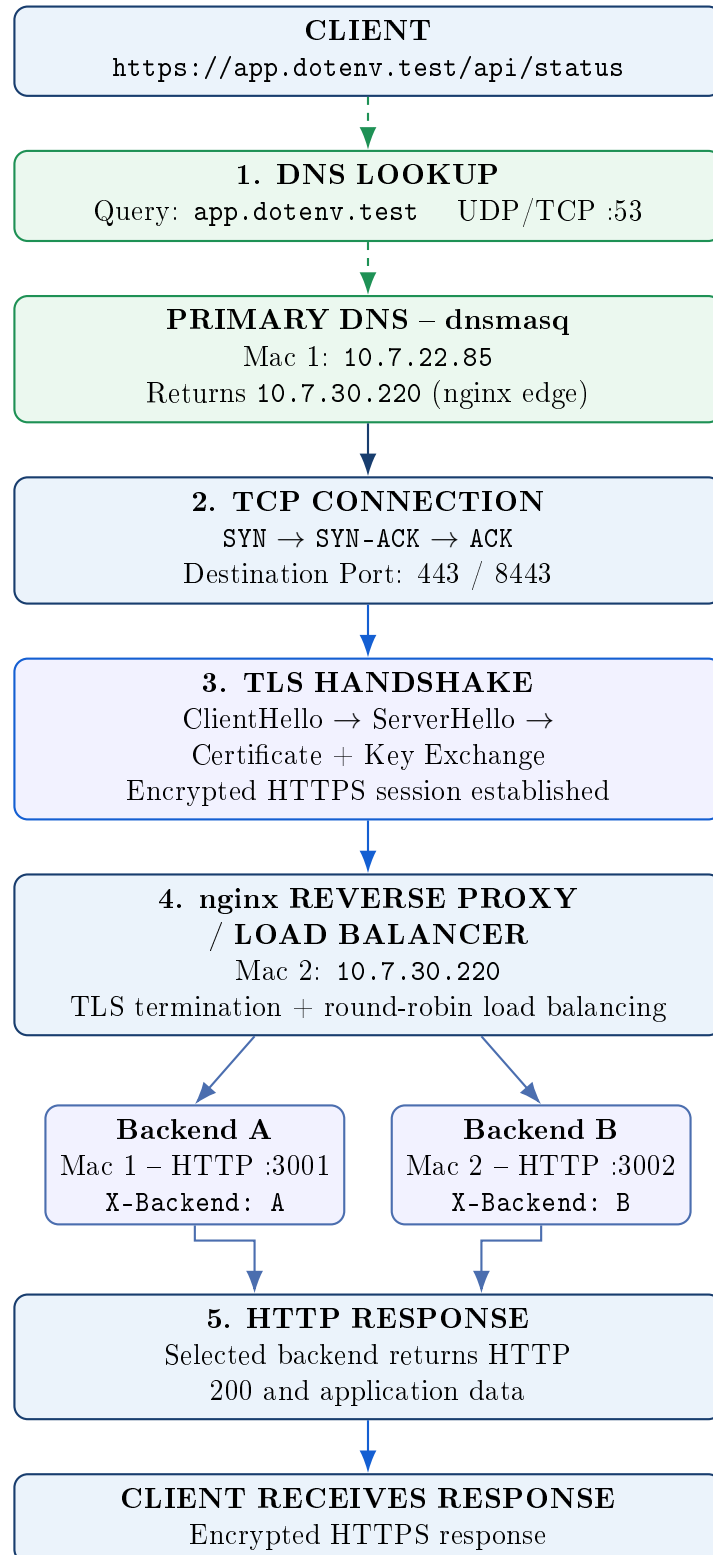


Figure 2: End-to-End Request Flow

3.1 Request Flow Explanation

1. The client requests `https://app.dotenv.test/api/status`.
2. The client performs a DNS lookup for `app.dotenv.test` using UDP/TCP port 53.
3. The private dnsmasq server returns the IP address of the nginx edge machine.
4. The client establishes a TCP connection using the three-way handshake: SYN, SYN-ACK, and ACK.
5. The TLS handshake establishes an encrypted HTTPS session between the client and nginx.
6. nginx terminates TLS and acts as the reverse proxy and load balancer.
7. nginx forwards the HTTP request to either Backend A or Backend B.
8. The selected backend generates the response and includes an identifying `X-Backend` response header.
9. nginx returns the response to the client through the encrypted HTTPS connection.

4 Protocol Layers

The request passes through the following protocol layers.

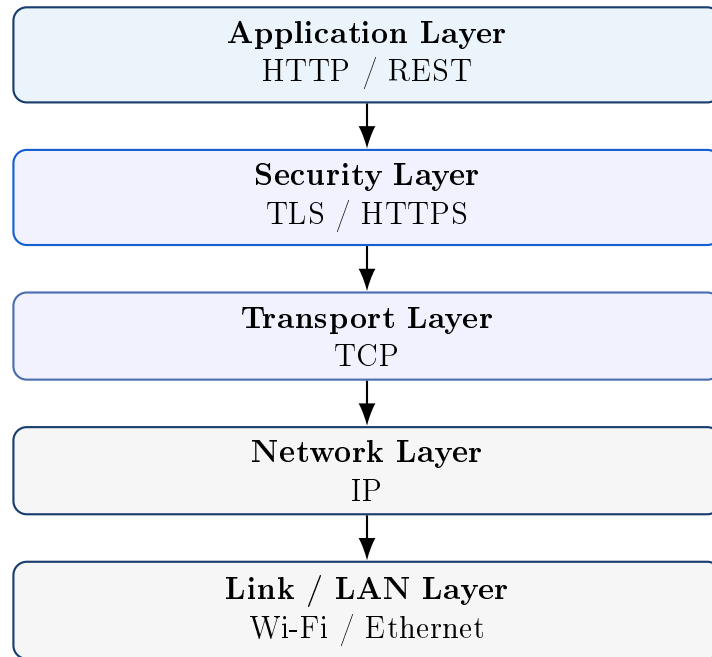


Figure 3: Protocol Layers Used in the Project

4.1 DNS

DNS provides the mapping between the service domain and the nginx edge IP address.

`app.dotenv.test` → `10.7.30.220`

DNS: UDP/TCP Port 53

4.2 TCP

After DNS resolution, the client establishes a reliable TCP connection with nginx.



The client uses an ephemeral source port and connects to the HTTPS destination port.

4.3 TLS / HTTPS

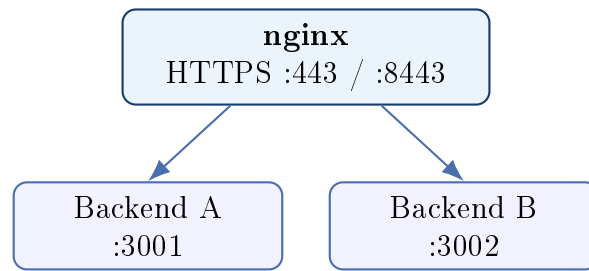
TLS provides encryption and authentication for the connection between the client and nginx. The handshake includes:

`ClientHello` $\rightarrow$ `ServerHello` $\rightarrow$ `Certificate` $\rightarrow$ `Key Exchange`

After the handshake, HTTP data is carried inside the encrypted HTTPS connection.

4.4 nginx Reverse Proxy and Load Balancing

nginx is the public entry point of the application and distributes requests between the two backend servers.



5 End-to-End Communication Summary



Client → DNS → TCP → TLS → nginx → Backend → nginx → Client